13.1 Introduction

Graphs are an incredibly important abstraction in computer science, so there has been a huge amount of work on algorithms for graphs. We'll spend much of the rest of the semester studying graph algorithms. Today we'll begin by talking about a few of the simplest, oldest, and (possibly) most useful algorithms.

Everyone should know what a graph is already, but in case you don't: a graph is a pair $G = (V, E)$ where V is a set of elements (usually called *vertices* or *nodes*) and $E \subseteq \binom{V}{2}$ is a set of edges. The notation $\binom{V}{2}$ means that the edges are unordered or *undirected*: the edge $\{u, v\}$ is the same as the edge $\{v, u\}$. We will sometimes talk about directed graphs, in which $E \subseteq V \times V$ and the edge (u, v) is different from the edge (v, u).

It is reasonably standard to let $n = |V|$ and $m = |E|$, so we're going to use that notation throughout.

13.2 Graph Representations

The first obvious question is: how do we represent a graph? What's the actual data structure we're going to use? There are many possibilities, but there are two classical answers: adjacency matrices and adjacency lists. Unless otherwise specified, we're going to assume that we use adjacency lists.

Adjacency Matrix: We have an $n \times n$ matrix A. We set $A_{ij} = 1$ if $\{i, j\} \in E$ and set $A_{ij} = 0$ otherwise.

Major benefits: can check whether an edge exists in constant time. It also turns out this matrix has really, really nice properties as a matrix. Obviously it is symmetric (for undirected graphs), but studying its eigenvalues and eigenvectors can actually tell us a huge amount about the graph. There's a whole area of graph theory called *spectral graph theory* which studies this.

Major drawbacks: takes $\Theta(n^2)$ space even if m is small. Iterating over the edges incident on a vertex v takes time $\Omega(n)$ even if there are only a few such edges (need to scan a whole row or column of the matrix).

Adjacency List: We have an array A of length n, and $A[v]$ is a pointer to a linked list which contains the edges incident on v. In other words, $A[v]$ is a linked list of the vertices that are adjacent to v.

Major benefits: only takes $O(n + m)$ space, can iterate over edges very efficiently.

Major drawbacks: takes a long time to check if $\{u, v\}$ is an edge (time $O(d(u))$ or $O(d(v))$, where $d(u)$ denotes the degree of u).

Adjacency Data Structures: Here's an obvious idea: let's replace the adjacency list $A[v]$ with something smarter than an adjacency list! For example, we could store all of the edges that were in the list in a red-black tree or a hash table (or, if the graph doesn't change, a two-level hash table using perfect hashing). This is basically a good idea, but we're not going to do it: lists are more traditional, and the algorithms that we're going to talk about don't benefit from being stored using fancier data structures. But it's a good option to keep in mind for the future.

13.3 Breadth-First Search

BFS is a way of traversing or searching a graph. It is one of the simplest and most obvious ways of doing this, and has a few nice properties.

Pseudocode is in CLRS – we suggest looking at it. But basically we start out from some node v. We mark it as complete, and put all of its neighbors in a queue (first-in first-out), marking them all as "in process". We then pull from the queue, mark that node as complete, and put its neighbors (that are not already in process or complete) in the queue (and mark them as "in process"). The key point is that we only put a node in the queue if it is not already in process or complete. This means that we essentially search by levels from v. We first see all of the neighbors of v, then we see all vertices at distance 2 from v, then all vertices at distance 3 from v, etc.

The running time is $O(n + m)$. We need to spend $\Theta(n)$ time on initialization, e.g. setting the "completed" and "in process" field of each node to 0. Once we've started the algorithm, note that the adjacency list of a vertex is scanned only when that vertex is pulled from the queue. Each node is pulled from the queue only once, so each adjacency list is scanned only once, so the time is only $O(m)$. Thus the total running time is $O(n + m)$.

A very nice feature of BFS is that it gives *shortest paths*. This is proved formally in the book, but let's give an informal argument here. Suppose we start at some node v. We claim that for any node u, the path in the BFS tree from v to u is the shortest possible path.

Let's do an informal proof by contradiction. Let $d(v, x)$ denote the length of the shortest path from v to x, for every $x \in V$. Suppose that u is the closest node to v in which the BFS does not return the shortest path. Let w' be the node just before u on the shortest path, and let w be the node just before u on the BFS path. If $w = w'$ then we are done: w' is closer to v than u is, so by assumption the BFS tree has the shortest path to w, and so it also has the shortest path to u, contradicting the definition of u. If $w \neq w'$, then we have two cases. If $d(v, w) = d(v, w')$ then we are done, since this means the BFS has a shortest path to u, again contradicting the definition of u. Otherwise $d(v, w') < d(v, w)$. But this means that the BFS would have found u from w' before it found u from w, giving a contradiction.

13.4 Depth-First Search

DFS is the other obvious way to explore a graph. Instead of going by breadth, we go by depth. Again, pseudocode is in the book, but we're going to spend a little more time on DFS since it will be useful in some of the algorithms we'll talk about later. DFS is super easy to implement recursively: when we call DFS on a node v, we iterate through its neighbors and for any unmarked

neighbor u we mark it and then call DFS on u. It can also be implemented iteratively using a stack rather than a queue.

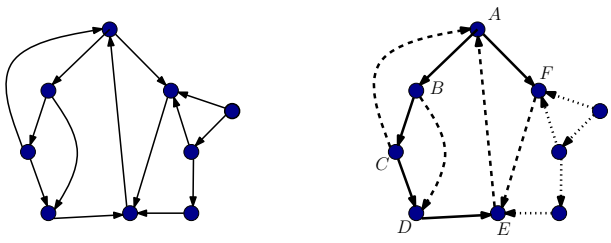
The basic pseudocode would look something like this. Let's assume that G is directed, since it's a more general setting and will be useful for the next few algorithms anyway.

```
Init: for each v in V, mark(v) = False

DFS(v) {
  mark(v) = True
  for each edge (v,u) in A[v] {
    if mark(u) == False then DFS(u)
  }
}
```

DFS also takes time $O(n+m)$. We again have to spend $\Theta(n)$ time initializing the graph, and then since we only visit each vertex once we only scan each adjacency list once, so we spend time $O(m)$ in the main algorithm.

Here's an example. The labeled nodes are the ones visited by calling DFS(A), the solid edges are traversed by the algorithm, the dashed edges were considered but not traversed, and the dotted edges were not even looked at.



Let's prove a simple property about DFS, which is obvious (and you should know from Data Structures) but is a good warmup proof.

Definition 13.4.1 *A node u is* reachable *from v if there is a path $v = v_0, v_1, \ldots, v_k = u$ such that each (v_i, v_{i+1}) is an edge of G.*

Theorem 13.4.2 *When DFS(v) terminates, it has visited (marked) all of the nodes that are reachable from v.*

Proof: We first prove that DFS(v) terminates. This is obvious because every call to DFS is to an unmarked node, and each such call immediately marks the node. Thus there can be at most n calls to DFS, so it terminates.

Now suppose that u is reachable from v but is not marked when DFS(v) terminates. Since u is reachable from v, there is a path $v = v_0, v_1, \ldots, v_k = u$ from v to u, and since v is marked but u is

not, there is some node v_i on this path which is the first unmarked node. But this is a contradiction, since when we examined the adjacency list of v_{i-1} we would have seen that v_i is unmarked and we would have recursively called DFS(v_i) and marked it. ■

Of course, it might not be possible to reach all nodes in the graph from our starting node, so it's pretty common to simply call DFS on unmarked vertices until everything has been marked:

```
DFS(G) {
  for all v in V, set mark(v) = false
  while there exists an unmarked node v
    DFS(v)
}
```

DFS naturally comes with a notion of timestamps. We can define start(v) to be the "time" at which we began processing v, and we can define finish(v) to be the "time" at which we finished processing v. We can modify our algorithm to keep track of these as follows.

```
DFS(G) {
  i = 0
  for all v in V {
    start(v) = 0
    finish(v) = 0
  }

  for all v in V if start(v) == 0 then DFS(v)
}

DFS(v) {
  i = i+1
  start(v) = i
  for each edge (v,u) in A[v] {
    if start(u) == 0 then DFS(u)
  }
  i = i+1
  finish(v) = i
}
```

This is clearly the same algorithm, it just keeps track of start and finish times, which are useful pieces of data when we try to reason about DFS and its uses.

13.4.1 Edge types

Define a *tree edge* to be an edge (v, u) such that DFS(v) calls DFS(u), i.e. the edge is traversed by the algorithm. It is easy to see that the tree edges actually form a collection of trees such that every node is in exactly one tree, i.e. they form a *spanning forest*. This is sometimes called the depth-first

forest (or tree) of G. Moreover, every tree is rooted at the node of the tree with minimum start time.

This DFS forest naturally lets us partition the edges into three types:

1. Forward edges are edges (v, u) where u is a descendant of v in this tree. This includes tree edges. Note that in any forward edge, $\text{start}(v) < \text{start}(u) < \text{finish}(u) < \text{finish}(v)$.
2. Back edges are edges (v, u) where u is an ancestor of v. Note that in any back edge, $\text{start}(u) < \text{start}(v) < \text{finish}(v) < \text{finish}(u)$.
3. Cross edges are edges (v, u) where u is neither an ancestor nor a descendant of v. In this case $\text{start}(u) < \text{finish}(u) < \text{start}(v) < \text{finish}(v)$.

A good exercise to do at home is to formally prove these relationships between the start and finishing times, but since they're reasonably intuitive we're just going to trust it for now (but you should know how to prove them!).